25/8/26 DAY 2:

Conditional Statements

Conditional statements are used to perform different actions based on different conditions.

if

- The if statement is used to test a condition. If the condition is true, the block of code inside the if statement is executed. If the condition is false, the block of code is skipped.

if-else

- The if-else statement is used to perform different actions based on a condition. If the condition is true, the block of code inside the if statement is executed. If the condition is false, the block of code inside the else statement is executed.

else-if

- The else-if statement is used to test multiple conditions. If the first condition is false, the program checks the next condition. This process continues until a true condition is found or all conditions are checked.

switch

- The switch statement is used to perform different actions based on different conditions.
- It is a efficient way to test multiple conditions than using multiple if-else statements.
- The switch statement evaluates an expression and executes the block of code corresponding to the matching case label.
- If no case matches, the default block is executed.

> When to use if-else and switch statements:

- Use if-else statements when you have a small number of conditions to check and the conditions and complexity of the conditions are not known in advance.
- Use switch statements when you have a large number of conditions to check and the conditions are known in advance. Switch statements are more efficient than if-else statements when there are many conditions to check.

```
#include <iostream>
using namespace std;

int main() {

    int age = 10;
    // single statement
    if(age>=18)
        cout << age << endl;
```

```
    // multi-line if with {}
    if(age >= 18){
        cout << "You can Vote!" << endl;
        cout << age;

    }

//   not recommended
    if(age){
        cout << "You can vote!!" << endl;
    }

    if(age >= 18){
        cout << "You can Vote!" << endl;
        cout << age;
    } else{
        cout << "You cannot Vote!" << endl;
    }

    int percentage = 55;

    // if-else ladder
    //  GRADE Checking
    if(percentage >= 70 && percentage <= 80){
        cout << "GRADE : A" << endl;
    } else if(percentage >= 60 ){
        cout << "GRADE : B" << endl;
    } else if(percentage >= 50){
        cout << "GRADE : C" << endl;
    } else{
        cout << "Fail!" << endl;
    }


    int choice = 1;

    switch(choice){
        case     1:
            cout << "Choice 1" << endl;
            break;  // jump statement
        case 2:
            cout << "Choice 2" << endl;
            break;
        case 3:
            cout << "Choice 3" << endl;
            break;
        default:
            cout << "Default" << endl;
    }

    return 0;
}
```

Loops

- Loops are written to perform a certain task repeatidly till a condition.

for

- It consists of three parts: initialization, condition, and increment/decrement.

while

- It consists of a condition that is checked before each iteration.

do-while

- It consists of a condition that is checked after each iteration.

```
  #include <iostream>
using namespace std;

int main() {

//  intialization
//  while(condition){
        //business logic code
        // increment/decrement
//  }

    // FOR LOOP
    // initialize; condition ; increment/decrement
    for(int i = 0; i<100 ; i++){
        cout << "FOR" << endl;
    }

    // DO-WHILE LOOP
    int i = 0;
    do{
        cout << "DO-WHILE" << endl;
        i++;
    }while(i<100);

    // WHILE LOOP
    int i = 0;
    while(i<100){
        cout << "WHILE" << i << endl;
        i++;
    }

    return 0;
}
```

Jump Statements

- Jump statements are used to transfer control to another part of the program.
- They are used to break out of loops or switch statements, or to return from a function.

break

- The break statement is used to exit a loop or switch statement before it has completed all its iterations.
- When the break statement is encountered, the program immediately exits the loop or switch statement and continues executing the code that follows it.

continue

- The continue statement is used to skip the current iteration of a loop and move on to the next iteration.
- When the continue statement is encountered, the program immediately jumps to the next iteration of the loop, skipping any remaining code in the current iteration.

return

- The return statement is used to exit a function and return a value to the calling function.

```
#include <iostream>
using namespace std;

int main() {

    for(int i=0;i<50;i++){
        if(i == 41){
            cout << "Value 40\n";
            break;  // move out of scope/iteration
        }

        if(i % 2 == 0)
            continue;   // skip the current iteration

        cout << i  << " ";
    }
    return 0;
}
```

Arrays

- Collection/Data Structure of similar data type stored in sequential manner in memory.
- The elements of an array are stored in contiguous memory locations, and each element can be accessed using its index.

Declaration and initialization of arrays

```
int arr[5]; // declaration of an array of size 5
int arr[5] = {1, 2, 3, 4, 5};   // declaration and initialization of an array of size 5
int arr[] = {1, 2, 3, 4, 5};    // declaration and initialization of an array of size 5
int arr[5] = {1, 2};    // declaration and initialization of an array of size 5, with the first two elements initialized to 1 and 2, and the rest initialized to 0

int arr[]= {1, 2, 3, 4, 5}; // valid
int arr[]{1, 2, 3, 4, 5};   // valid
```

1-D arrays

```
int arr[5] = {1, 2, 3, 4, 5};   // declaration and initialization of a 1-D array of size 5
```

2-D arrays

```
int arr[2][2] = {{1, 2}, {4, 5}};   // declaration and initialization of a 2-D array of size 2x2
```

Array traversal and processing

```
    int arr[5] = {1, 2, 3, 4, 5};   // declaration and initialization of an array of size 5

    for(int i=0; i<5; i++){
        cout << arr[i] << " ";  // traversal of an array
    }
```

```
#include <iostream>
using namespace std;

int main(){

    int size;

    cout << "Enter the Size of Array: " << endl;
    cin >> size;

    int arr[size];
```

```
    cout << "Enter the Elements of Array: " << endl;

    for(int i=0;i<size;i++){
        cin >> arr[i];
    }

    cout << "Array Elements: ";
    for(int i=0;i<size;i++){
        cout << arr[i] << " ";
    }

//  for(int i=0;i<5;i++){
//      if(arr[i] == 40){
//          cout << "Found";
//          return 0;
//      }
//  }
//  cout << "Not Found";

    return 0;
}

int main2(){

    int arr[]{10,20,30,40,50};

    for(int i=0;i<5;i++){
        if(arr[i] == 40){
            cout << "Found";
            return 0;
        }
    }
    cout << "Not Found";

    return 0;
}

int main1() {

    // variables
    int val = 10;   // initialization
    int val1(10);   // initialization
    int val2{10};   // initialization

//  cout << val << endl;
//  cout << val1 << endl;
//  cout << val2 << endl;

    int arr[10];    // create a array of type int   declaration + definition

    int arr1[3] = {10,30,27};   // initialization

    int arr2[]{20,39,78};       //initialization
```

```
//  cout << arr2[2];

    cout << arr1 << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {

    int arr[2][2];  //declaration + definition of 2d array

    int arr1[2][2]{
            {10,10},{20,20}
    };

    cout << arr1[0] << endl;
    cout << arr1[1] << endl;

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout << arr1[i][j] << " ";
        }
    }

    return 0;
}
```

Command-line arguments

- Command-line arguments are used to pass information to a program when it is executed.
- They allow users to provide input to the program without having to modify the code or use input prompts.
- Command-line arguments are typically passed as strings, and they can be accessed using the **argc** and **argv** parameters of the **main** function.

argc and argv

argc

- **argc** is an integer that represents the number of command-line arguments passed to the program, including the name of the program itself.

argv

- **argv** is an array of strings that contains the actual command-line arguments passed to the program.

```cpp
#include <iostream>
using namespace std;

int main(int argc, char* argv[]) {

//  argc -> argument count : stores the count of arguments from command line in form of int
//  argv -> argument vector: stores all the values coming from command line in form of string

//  cout << "Total Arguments: " << argc << endl;
//  cout << "Total Arguments: " << argv[0] << endl;

    cout << "Sum from CommandLine: " << stoi(argv[1]) + stoi(argv[2]) << endl;

//  ./Day2.7.exe        : argument

    return 0;
}
```

Functions

- A named block of code which is design for perfoming a perticular task.
- Functions are used to break down a program into smaller, more manageable pieces, making it easier to read, understand, and maintain.
- They also allow for code reuse, as the same function can be called multiple times from different parts of the program.

```cpp
returnType functionName(parameter1, parameter2, ...) {
    // function body
    // return statement (if applicable)
}
```

Function declaration

- A function declaration is a statement that tells the compiler about the function's name, return type, and parameters.

```cpp
int functionName(int param1, int param2);
```

Function definition

```cpp
int functionName(int param1, int param2) {
    // function body
```

```
}
```

Function prototype

- It is same as function declaration.

```
int functionName(int, int); // function prototype = function declaration
int functionName(int param1, int param2); // function prototype = function
declaration
```

Function call

- A function call is a statement that invokes a function and passes any required arguments to it.

```
int result = functionName(10, 20);
```

Different forms of functions

1. With parameters and with return type

```
int add(int a, int b) {
    return a + b;
}
```

2. With parameters and without return type

```
void printSum(int a, int b) {
    cout << "Sum: " << a + b << endl;
}
```

3. Without parameters and with return type

```
int getSum() {
    return 10 + 20;
}
```

4. Without parameters and without return type

```
void printValues() {
    cout << "Values: 10, 20" << endl;
```

```
}
```

Tomorrow we will discuss about:

Function Again...

Call by Value

Call by Reference

Reference variable

Reference syntax

Reference vs Pointer

Inline Functions

Math library functions